

Operating Systems – Lecture 1: Introduction to Operating Systems

What is an Operating System (Sistemi Operativ)?

- **Definition:**
An Operating System (OS) is system software that manages hardware and software resources and acts as an intermediary between users and the hardware.
 - **Analogy:**
👉 Think of it like a manager in a company—without one, employees (hardware) wouldn't know what tasks to do, and clients (users) wouldn't get their work done.
-

Core Responsibilities of an OS

- **Manages Processes:**
✓ Decides which programs run and for how long.
- **Controls Memory:**
✓ Allocates and tracks RAM usage.
- **Handles Input/Output (I/O):**
✓ Manages files, devices, and communication.
- **Enforces Security:**
✓ Restricts unauthorized access.
- **Provides a User Interface:**
✓ Offers a GUI (Graphical User Interface) or CLI (Command Line Interface).

💡 Example:

When you open Google Chrome:

- The OS allocates memory to Chrome.
 - It schedules CPU time so the browser runs smoothly.
 - It handles I/O requests (mouse clicks, typing).
 - It secures network access to prevent unauthorized connections.
-

Why Do We Need an OS?

- A computer **without an OS** is just a pile of circuits.
- The OS **coordinates everything**, making it possible to run multiple programs at once without conflict.

💡 Example:

Imagine you're running Spotify + Word + YouTube:

- The OS ensures that Spotify doesn't freeze Word.

- It shares CPU time between tasks.
 - It prioritizes processes so that an urgent task (like a keyboard press) happens instantly.
 - Without an OS, every program would have to manage hardware manually—which is impractical.
-

The 4 Main Components of a Computer System

1. 🖨️ **Hardware (Hardueri):**
The physical components like CPU, RAM, and disk.
2. ⚙️ **Operating System (Sistemi Operativ):**
Manages and controls everything.
3. 📁 **Application Programs (Programet Aplikative):**
Software that users interact with (e.g., browsers, games).
4. 👤 **Users (Shfrytëzuesit):**
The people using the system.

💡 Example:

A Linux server running a website:

- The OS schedules HTTP requests.
 - Memory management prevents crashes.
 - File system operations ensure smooth database storage.
-

The Kernel – The Brain of the OS

- **Definition:**
The kernel is the core component of the OS that is always running and controls the most critical operations.
- **Types of Kernels:**
 - **🔹 Monolithic Kernel (e.g., Linux, Windows):**
Everything runs together—fast but harder to maintain.
 - **🔹 Microkernel (e.g., QNX, Minix):**
Only essential functions run in the kernel while other services run separately—more stable but slower.

💡 Example:

A Windows system crash (Blue Screen of Death) is often due to a kernel failure, meaning the core system became unstable.

The Boot Process – How a Computer Starts Up

1. **BIOS/UEFI:**
📁 Loads firmware stored in ROM.
2. **Bootstrap Program:**
🔧 Initializes the OS using a bootloader (like GRUB or Windows Boot Manager).
3. **Kernel Loading:**
📦 The kernel is loaded into RAM and system services start running.
4. **User Interaction:**
🖱️ The system is ready for user input.

💡 **Example:**

If a system doesn't boot, it might be due to corrupt bootloader files, requiring recovery.

Storage Hierarchy – Managing Data Efficiently

- 🚀 **Registers (CPU L1/L2 Cache):**
Fastest storage; holds CPU instructions for immediate execution.
- 💾 **RAM (Memory):**
Fast but volatile; used for temporary program execution.
- 🕒 **SSD (NVMe, SATA):**
Fast, non-volatile storage where the OS and software are installed.
- 🕒 **HDD (Magnetic Disks):**
Slower than SSDs; used for large-scale file storage.
- ☁️ **Cloud/External Storage:**
Slowest but permanent; used for backups and remote access.

💡 **Example:**

When you copy a file from an SSD to a USB, you notice a speed difference due to the different data transfer rates among storage types.

Caching – Speeding Up Performance

- **Purpose:**
Caching stores frequently accessed data in faster memory to reduce wait times.
- **Types of Caching:**
 - 💎 **CPU Cache (L1/L2):** Speeds up instruction execution.
 - 💎 **Disk Cache:** Preloads file system data for quick retrieval.
 - 💎 **Browser Cache:** Saves websites for faster loading.

💡 Example:

When you visit a website, the browser stores static content (images, styles) locally so it loads faster next time.

Device Drivers – How OS Communicates with Hardware

- **Function:**
Device drivers translate OS commands into hardware-specific operations.
- **Types of Drivers:**
 - **Character Drivers:** Handle devices like keyboards, mice, and serial ports.
 - **Block Drivers:** Manage storage devices (HDD, SSD).
 - **Network Drivers:** Control network interfaces (WiFi, Ethernet, Bluetooth).

💡 Example:

When you plug in a new printer, the OS installs a driver to recognize and use it properly.

Process Management – Handling Multiple Programs at Once

- **Definition:**
A process is a program in execution, and the OS manages these processes efficiently.
- **Process Life Cycle:**
 - ➊ **New:** Process is created.
 - ➋ **Running:** Actively using the CPU.
 - ➌ **Waiting:** Suspended for I/O or an event.
 - ➍ **Ready:** Waiting for CPU allocation.
 - ➎ **Terminated:** Process completes execution.

💡 Example:

When you watch a video while downloading a file, the OS ensures both tasks run smoothly by switching CPU time between them.

Virtualization – Running Multiple OS on One System

- **Concept:**
Virtualization allows multiple OS instances to run on the same physical machine.
- **Types of Virtualization:**
 - **Hypervisors (e.g., VMware, KVM, Xen):** Create full virtual machines (VMs).
 - **Containers (e.g., Docker, Kubernetes):** Provide isolated environments sharing the same OS kernel.

💡 Example:

Cloud platforms like AWS and Google Cloud run thousands of VMs using hypervisors, allowing multiple users to share resources without interference.

Why This Matters

- **Optimizing System Performance:**
The OS manages resources efficiently.
- **Troubleshooting Issues:**
Understanding OS structure helps diagnose boot errors and system crashes.
- **Software Development:**
Developers can build applications that interact seamlessly with system resources.

Operating Systems – Lecture 2: Structure of Operating Systems

Services Provided by the Operating System

- **User Interface (UI):**
📄 Almost all OS provide a user interface—either a Graphical User Interface (GUI) (e.g., Windows, macOS) or a Command-Line Interface (CLI) (e.g., Linux Terminal).
- **Program Execution:**
⚙️ The OS loads programs into memory, executes them, and terminates them (either normally or due to errors such as segmentation faults).
- **I/O Operations:**
📁 Handles file access and I/O device communication (keyboards, printers, USBs) so programs don't need to directly control hardware.
- **File System Management:**
📁 Organizes files and directories, managing file permissions and access control.
- **Process Communication:**
🔗 Provides Inter-Process Communication (IPC) mechanisms (e.g., shared memory, message passing) for data exchange between applications.
- **Error Detection and Handling:**
🚨 Monitors system components (CPU, memory, I/O devices) and takes corrective actions when failures occur.
- **Resource Allocation:**
🔄 Distributes CPU time, memory, and I/O bandwidth efficiently among multiple processes.
- **Logging (Accounting):**
📊 Keeps track of resource usage by users and processes for monitoring and auditing.

- **Protection & Security:**

🔒 Implements user authentication (passwords, biometrics) and enforces access controls to prevent unauthorized use.

💡 **Example:**

When multiple users access a Linux server, the OS enforces file permissions (read/write/execute) so that one user cannot modify another user's files.

System Calls – How Applications Interact with the OS

- **Definition:**

System calls serve as the interface between user applications and the OS kernel. Programs use system calls to perform tasks like file operations, memory allocation, or process control without directly accessing hardware.

- **System Call Workflow:** 1. A program requests a service by calling a system function.

2. The system call is mapped to a unique number and forwarded to the OS kernel.

3. The OS executes the requested operation (e.g., opening a file).

4. The result is returned to the application.

💡 **Example:**

When a Python script opens a file using `open("data.txt", "r")`, the OS executes a system call (like `sys_open()`) to handle the file access at a lower level.

Common OS APIs for System Calls

- 🚀 **Win32 API:**

Used for system calls in Windows OS.

- 🚀 **POSIX API:**

Standard for UNIX-based systems (e.g., Linux, macOS).

- 🚀 **Java API:**

Used in JVM-based applications to provide cross-platform functionality.

💡 **Example:**

A C program on Linux calls `fork()`, defined in the POSIX API, to create a new process at the kernel level.

Passing Parameters in System Calls

- **Methods to Pass Parameters:** 1. **Registers:**

Parameters are placed in CPU registers (fastest method). 2. **Memory Blocks:**

Parameters are stored in memory, and only the address is passed. 3. **Stacking:**

Parameters are pushed onto the process stack and retrieved when needed.

💡 Example:

When calling `write(fd, buffer, size)`, the OS may place `fd` in a register, store `buffer` in memory, and push `size` onto the stack.

Types of System Calls

- **Categories:**

-  **File Management:**
Create, open, read, write, and delete files.
-  **Device Management:**
Communicate with devices (e.g., printers, hard drives).
-  **Information Maintenance:**
Retrieve or set system information (e.g., `getpid()`).
-  **Process Communication:**
Send and receive messages between processes.
-  **Protection & Security:**
Manage user access and file permissions.

💡 Example:

The `exec()` system call replaces a running process with a new program, which is commonly used in command-line shells.

System Programs – Essential OS Tools

- **Definition:**
System programs are user-level utilities that interact with the OS kernel to provide a friendly environment for development and system management.
- **Examples of System Programs:**
 -  **File Manipulation:**
Commands like `mv` or `rm` in Linux for copying, renaming, or deleting files.
 -  **Status Information:**
Tools such as `top` or Task Manager display system statistics.
 -  **Programming Support:**
Compilers and debuggers (e.g., GCC, Visual Studio).
 -  **Program Execution:**
Shells (e.g., `bash`, `cmd.exe`) for launching applications.
 -  **Inter-Process Communication:**
Programs that manage network connections.

-  **Background Services:**
Daemons (in Linux) or Windows Services that run background tasks.

 Example:

In Linux, the `grep` command is a system program used for searching text within files. It relies on system calls like `open()` and `read()` to access file data.

How Programs Are Compiled and Linked

- **Steps Involved:**
 - 1 Relocatable Object File:**
After compilation, source code is converted into object files that can be loaded into memory at different locations.
 - 2 Linker:**
Combines multiple object files into a single binary executable and links necessary libraries.
 - 3 Loader:**
The OS loads the binary executable into memory and prepares it for execution.

 Example:

When you compile a C program using `gcc main.c`, the linker combines the standard libraries (e.g., `libc`) with your code before execution.

Memory Relocation & ABI (Application Binary Interface)

- **Memory Relocation:**
During execution, the OS assigns final memory addresses for code and data.
- **ABI:**
Defines how compiled programs interact with the OS and hardware, ensuring compatibility across different architectures (e.g., x86 vs. ARM).

 Example:

An executable compiled for Windows using the x86-64 ABI will not run on macOS with an ARM64 ABI without recompilation.

Design & Implementation Goals of an OS

- **User Goals:**
The OS should be easy to use, secure, and fast.
- **System Goals:**
The OS should be flexible, reliable, and efficient while minimizing errors.

 Example:

Windows prioritizes user experience, whereas Linux is optimized for stability and efficiency.

Policy vs. Mechanism in OS Design

- **Policy:**
Defines *what* should be done (e.g., prioritizing real-time processes).
- **Mechanism:**
Defines *how* it should be done (e.g., implementing a Round-Robin scheduling algorithm).

💡 **Example:**

In CPU scheduling, the policy determines which process gets priority, while the mechanism implements the actual algorithm.

UNIX System Structure

- **Components:**
 - **◆ System Programs:**
User-level utilities and command-line tools.
 - **◆ Kernel:**
Provides system calls, file system management, CPU scheduling, and memory management.
-

Layered OS Architecture

- A layered OS is constructed in multiple levels:
 - **Lowest Layer (Layer 0):**
Directly interacts with hardware.
 - **Highest Layer:**
Provides the User Interface (UI).
-

Microkernel Architecture

- **Concept:**
In microkernel systems, only essential functions (e.g., process communication, memory management) run in kernel mode. Other services run in user mode.
- **Advantages:**
Improved stability and modularity.
- **Examples:**
macOS and Windows NT use hybrid microkernel designs for enhanced performance.

Operating Systems – Lecture 3: Processes

What is a Process?

- **Definition:**
A process is an instance of a program in execution.

- **Components of a Process:**

- **Code (Text Section):** Contains the actual instructions.
- **Program Counter:** Keeps track of the next instruction to execute.
- **Stack:** Stores function calls and local variables.
- **Heap:** Stores dynamically allocated memory during runtime.
- **Data Section:** Contains global variables.

 Example:

When you open Google Chrome, each tab is treated as a separate process so that if one tab crashes, it doesn't affect the others.

Process Control Block (PCB) – The Identity of a Process

- **Definition:**

The PCB stores critical information needed by the OS to manage a process.

- **Contents of a PCB:**

- **Process State:** New, running, waiting, ready, or terminated.
- **Program Counter:** Tracks the next instruction.
- **CPU Registers:** Holds current register values.
- **CPU Scheduling Information:** Includes priority and queue pointers.
- **Memory Management Info:** Memory allocated to the process.
- **Accounting Information:** Tracks CPU usage and execution time.
- **I/O Status Information:** Lists open files and assigned I/O devices.

 Example:

If a process is suspended, its PCB stores all its execution details so that it can resume later without loss.

Context Switching – How the OS Switches Between Processes

- **Definition:**

The process by which the CPU switches from executing one process to another.

- **Steps:** 1. Save the state (registers, program counter) of the current process in its PCB. 2. Load the PCB of the next process to be executed. 3. Resume execution from where the process was paused.

 Example:

If you're typing in Word while listening to music, the OS rapidly switches between Word and Spotify, making both applications appear to run simultaneously.

Process Execution in Mobile OS (iOS vs. Android)

-  **iOS:**
Only one foreground process runs at a time, controlled by the UI. Background processes exist but are heavily restricted to save battery.
-  **Android:**
Runs both foreground and background processes with fewer restrictions. Background processes can continue using services even when not visible.

Example:

On iOS, Spotify stops playing if it lacks background audio permissions, whereas on Android, a music player can continue playing without UI activity.

Process Creation & Termination

- **Process Creation:**
 - A parent process spawns child processes, forming a process tree.
 - Each process is uniquely identified by a Process Identifier (PID).
- **Process Termination:**
 - A process calls `exit()` upon completion.
 - A parent process can terminate a child using commands such as `abort()`.
 - If a parent is terminated, the OS might also terminate its child processes.

Example:

In Linux, when you open VS Code, it creates multiple child processes for rendering, extensions, and debugging. When VS Code is closed, all its child processes terminate automatically.

Google Chrome's Multi-Process Architecture

- **Components:**
 - **Browser Process:**
Handles the main UI, network requests, and tab management.
 - **Renderer Process:**
Runs web pages and executes JavaScript in a sandboxed environment.
 - **Plug-in Process:**
Handles third-party plugins (e.g., Flash, though deprecated).

Example:

If one tab crashes, only the renderer process for that tab is affected—not the entire browser.

Independent vs. Cooperating Processes

- **Independent Processes:**
Do not affect or interact with one another.
- **Cooperating Processes:**
Share data and depend on each other's execution.

💡 Example:

- **Independent:** Playing a video while downloading a file.
 - **Cooperating:** A compiler's stages (parsing, optimization, code generation) that depend on each other.
-

Interprocess Communication (IPC) – How Processes Communicate

- **Models of IPC:**
 - **Shared Memory:**
Processes share a common memory space to exchange data (fast but requires synchronization).
 - **Message Passing:**
Processes send and receive messages through the OS (slower but more secure).

💡 Example:

A web server (Apache) may use shared memory for fast caching, whereas email services might use message passing for reliable communication.

Producer-Consumer Problem – A Common IPC Scenario

- **Concept:**
A Producer Process creates data, and a Consumer Process uses it.
- **Buffer Types:**
 - **Unbounded Buffer:**
The producer never waits (no limit on storage).
 - **Bounded Buffer:**
The producer must wait if the buffer is full, and the consumer waits if the buffer is empty.

💡 Example:

- **Unbounded:** A log file that continuously writes data.
 - **Bounded:** A print queue that processes jobs one at a time.
-

Message Passing in IPC

- **Direct Communication:**
Processes explicitly specify each other when sending and receiving messages.
 - **Commands:**
 - `send(P, message)` – Send a message to process P.
 - `receive(Q, message)` – Receive a message from process Q.
- **Indirect Communication:**
Messages are placed in a mailbox (queue) with a unique ID; processes retrieve messages from the mailbox.

 Example:

A chat application may use direct communication, whereas an email server stores messages in a mailbox until the recipient checks them.

Blocking vs. Non-Blocking Communication

- **Blocking (Synchronous):**
The sender waits until the message is received.
 - **Blocking Send:** Sender pauses until the message is received.
 - **Blocking Receive:** Receiver pauses until a message arrives.
- **Non-Blocking (Asynchronous):**
The sender sends the message and continues execution immediately.
 - **Non-Blocking Send:** Sender does not wait.
 - **Non-Blocking Receive:** Receiver either gets a valid message or an empty response.

 Example:

- **Blocking:** Like a phone call where the caller waits for the receiver to pick up.
 - **Non-Blocking:** Sending a text message, where the sender continues with other tasks.
-

Buffering – How Messages Are Stored Before Delivery

- **Buffer Types:**
 - **Zero Capacity Buffer:**
No buffering; the sender waits until the receiver is ready.
 - **Bounded Capacity Buffer:**
The buffer has a limited size; if full, the sender must wait.
 - **Unbounded Capacity Buffer:**
The buffer can grow indefinitely; the sender never waits.

 Example:

A printer queue is a bounded buffer—if too many print jobs are queued, new jobs must wait.

Pipes – Simple Data Transfer Between Processes

- **Definition:**
A pipe is a unidirectional communication channel between processes.
- **Types:**
 - **Ordinary Pipes:**
Exist only between a parent and child process.
 - **Named Pipes (FIFOs):**
Can be used between unrelated processes.

💡 Example:

In Linux, the command `ls | grep "file"` creates a pipe, passing the output of `ls` as input to `grep`.

Sockets – Network Communication Between Processes

- **Definition:**
A socket is an endpoint for network communication that allows processes on different computers to exchange data.

💡 Example:

When you browse the web, your browser opens a socket connection to a web server, sending an HTTP request and receiving a response.

Operating Systems – Lecture 4: Threads, Concurrency, and CPU Scheduling

Concurrency vs. Parallelism

- **Concurrency:**
 - ◆ Supports multiple tasks making progress simultaneously, though not necessarily executing at the same moment.
- **Parallelism:**
 - ◆ Multiple tasks are executed at the same time on different CPU cores.

💡 Example:

- **Concurrency:** A single-core processor rapidly switches between tasks, giving the appearance of simultaneous execution.
 - **Parallelism:** A multi-core processor actually runs several tasks at the same time.
-

Types of Parallelism

- **Data Parallelism:**
 - ◆ Splits large data into smaller chunks, distributing them across multiple cores to perform the same operation concurrently.

- **Task Parallelism:**
 - ◆ Divides the computation into distinct tasks, with each task handled by a separate thread.

💡 Example:

- **Data Parallelism:** Processing an image where each core processes a different portion of the pixels.
 - **Task Parallelism:** In a web server, one thread handles incoming requests while another manages database queries.
-

User Threads vs. Kernel Threads

- **User Threads:**
 - ◆ Managed by user-space libraries; the OS kernel is unaware of them.
 - **Examples:** POSIX Pthreads, Windows Threads, Java Threads.
 - **Advantage:** Faster context switching; lighter overhead.
 - **Kernel Threads:**
 - ◆ Managed directly by the OS kernel, allowing better use of multi-core CPUs.
 - **Disadvantage:** Higher overhead due to kernel involvement.
-

Multithreading Models

- **Many-to-One Model:**
 - ☐ Multiple user threads are mapped to a single kernel thread.
 - **Limitation:** If one thread blocks, the entire process blocks.
- **One-to-One Model:**
 - ☑ Each user thread maps to its own kernel thread.
 - **Advantage:** True parallel execution on multi-core systems.
- **Many-to-Many Model:**
 - ☐ Multiple user threads are mapped to a pool of kernel threads.
 - **Advantage:** Balances performance with resource efficiency.

💡 Example:

- **Many-to-One:** Early Java threads (limited by a single kernel thread).
 - **One-to-One:** Windows threads or Linux pthreads.
 - **Many-to-Many:** Modern Linux threading models that use hybrid thread pools.
-

CPU Scheduling – How the OS Decides Which Process Runs

- **Definition:**
A CPU scheduler selects processes from the ready queue and assigns them to the CPU.
 - **When Scheduling Occurs:**
 - 1 When a process moves from running to waiting (e.g., waiting for I/O).
 - 2 When a process is preempted and moves from running to ready.
 - 3 When a waiting process becomes ready (after I/O completion).
 - 4 When a process terminates.
-

Scheduling Strategies

- **Non-Preemptive Scheduling:**
 - ◆ Once a process gets the CPU, it runs until it voluntarily releases it or terminates.
- **Preemptive Scheduling:**
 - ◆ The OS can force a process to give up the CPU if a higher-priority process becomes available.

💡 Example:

- **Non-Preemptive:** A cashier serving one customer completely before moving to the next.
 - **Preemptive:** An emergency room treating critical patients first regardless of arrival time.
-

The Dispatcher – Switching Processes

- **Role:**
The dispatcher is responsible for handing over the CPU to the next scheduled process.
- **Key Tasks:**
 - ✓ Save the state of the current process.
 - ✓ Load the state of the next process.
 - ✓ Switch execution to the new process.
- **Dispatch Latency:**
 - ◆ The time taken by the dispatcher to complete the process switch.

💡 Example:

A fast dispatcher leads to smoother multitasking and reduced CPU idle time.

Key CPU Scheduling Criteria

- **CPU Utilization:**
 - ◆ Ensuring the CPU is busy as much as possible.
- **Throughput:**
 - ◆ Maximizing the number of processes completed per unit time.

- **Turnaround Time:**
 - ◆ Total time from process creation to completion.
- **Waiting Time:**
 - ◆ Time a process spends in the ready queue before execution.
- **Response Time:**
 - ◆ Time between a process's request and the first response.

💡 Example:

- Real-time systems prioritize low response time.
- Batch systems focus on high throughput.

Common CPU Scheduling Algorithms

- 📌 **First-Come, First-Served (FCFS):**
 - ◆ Processes are executed in the order they arrive.
 - **Type:** Non-preemptive.
 - **Problem:** Convoy effect if a long process arrives first.
- 📌 **Shortest Job First (SJF):**
 - ◆ Processes with the shortest burst time are executed first.
 - **Can be:** Preemptive (Shortest Remaining Time First) or non-preemptive.
 - **Problem:** Starvation for longer processes.
- 📌 **Round Robin (RR):**
 - ◆ Each process gets a fixed time slice (quantum), usually between 10-100ms.
 - **Behavior:** If a process doesn't finish within its time slice, it moves to the back of the queue.
- 📌 **Priority Scheduling:**
 - ◆ Processes are assigned priorities; the highest-priority process runs first.
 - **Can be:** Preemptive or non-preemptive.
 - **Problem:** Starvation for low-priority processes (can be mitigated by aging).
- 📌 **Multilevel Queue Scheduling:**
 - ◆ The ready queue is divided into multiple queues, each with its own priority level.
 - **Characteristic:** Once a process is assigned to a queue, it remains there.
- 📌 **Multilevel Feedback Queue (MLFQ):**
 - ◆ Similar to multilevel queue scheduling, but processes can move between queues based on their behavior.

- **Characteristic:** New processes start at a high priority and are moved down if they use too much CPU, while long-waiting processes can be promoted to avoid starvation.

💡 **Example:**

A background task like file indexing might start with low priority but be moved up if it hasn't executed for a long time.

Operating Systems – Lecture 5: Process Synchronization

Why Process Synchronization is Important

- **Purpose:**
In a multitasking environment, multiple processes may access shared resources simultaneously.
- **Risk:**
Without proper synchronization, race conditions and data inconsistencies can occur.

💡 **Example:**

- **Banking System:** Two processes updating the same bank account concurrently may cause an incorrect balance.
- **File System:** Two processes writing to the same file without coordination can lead to data corruption.

The Critical Section Problem

- **Definition:**
A critical section is a part of a process's code that accesses shared resources. Only one process should execute in its critical section at a time.
- **Structure of a Process with a Critical Section:**
 - 1 **Entry Section:**
The process requests access to the critical section.
 - 2 **Critical Section:**
The process executes code that modifies shared data.
 - 3 **Exit Section:**
The process leaves the critical section, allowing others to enter.
 - 4 **Remainder Section:**
The process performs non-critical operations.
- **Requirements for a Solution:**
 - ✓ **Mutual Exclusion:** Only one process can be in the critical section at any time.
 - ✓ **Progress:** If no process is in the critical section, any waiting process should eventually enter.
 - ✓ **Bounded Waiting:** A process should not wait indefinitely to enter the critical section.

💡 Example:

If two people want to withdraw money from the same ATM, the system must ensure that only one transaction occurs at a time.

Hardware Synchronization Mechanisms

- **Locks & Interrupts:**
 - **Single-Processor Systems:**
Disabling interrupts to prevent context switches during critical operations.
 - **Multi-Processor Systems:**
Use atomic hardware instructions to enforce synchronization.
- **Atomic Operations:**
 - **Test-and-Set (TAS):**
Used to implement locks on shared resources.
 - **Compare-and-Swap (CAS):**
Ensures that a value is changed only if it has not been modified by another process.

💡 Example:

A spinlock employs atomic instructions so that multiple threads do not modify shared data simultaneously.

Mutex Locks – Ensuring Mutual Exclusion

- **Definition:**
A mutex (mutual exclusion) is a binary lock that restricts access to a shared resource so that only one process can hold the mutex at a time.
- **Operations:**
 - **acquire():**
Locks the mutex. If another process holds it, the calling process waits.
 - **release():**
Unlocks the mutex, allowing another process to enter the critical section.

💡 Example:

A printer spooler uses a mutex to ensure that only one print job is processed at a time.

Semaphores – Controlling Access to Shared Resources

- **Definition:**
A semaphore is a counter used to manage access to a shared resource by multiple processes.
- **Types:**

- **Binary Semaphore (Mutex):**
Can have only two values: 0 (locked) and 1 (unlocked).
- **Counting Semaphore:**
Allows multiple processes to access a resource concurrently.
- **Operations:**
 - **wait() / P():**
Decrements the semaphore count; if the count is zero, the process waits.
 - **signal() / V():**
Increments the semaphore count, possibly unblocking waiting processes.

 Example:

- **Database Connection Pooling:** A semaphore limits the number of active database connections.
- **Traffic Light System:** Controls how many cars can pass at a given time.

Monitors – High-Level Synchronization Constructs

- **Definition:**
A monitor is a synchronization construct that groups shared variables, procedures, and synchronization primitives into a single module.
- **Characteristics:**
 - Only one thread can execute inside a monitor at a time.
 - Threads wait until a certain condition is met.
 - Automatic mutual exclusion is enforced, eliminating the need for explicit locks.

 Example:

A hotel reservation system using monitors ensures that only one customer can book a room at a time, preventing double bookings.

Differences Between Semaphores and Monitors

- **wait() Behavior:**
 - **Semaphores:**
If wait() is called and the count is positive, no blocking occurs.
 - **Monitors:**
Calling wait() causes the process to block until the condition is met.
- **signal() Behavior:**
 - **Semaphores:**
Increments the semaphore count and unblocks a waiting process if necessary.

- **Monitors:**
Wakes up a waiting process without modifying the condition variable.

💡 **Example:**

In a producer-consumer problem, monitors ensure that the producer waits if the buffer is full and the consumer waits if the buffer is empty.

Blocking vs. Non-Blocking Synchronization

- **Blocking (Synchronous):**
The process waits until a condition is met before continuing execution.
- **Non-Blocking (Asynchronous):**
The process continues execution immediately and checks the condition later.

💡 **Example:**

- **Blocking:** A thread waits for a file to finish downloading before processing it.
 - **Non-Blocking:** A thread initiates a file download and continues executing, checking later if the file is ready.
-

Buffering – How Messages Are Stored in Synchronization

- **Buffer Types:**
 - **Zero Capacity Buffer:**
The sender must wait until the receiver is ready.
 - **Bounded Capacity Buffer:**
The buffer has a fixed size; if it's full, the sender must wait.
 - **Unbounded Capacity Buffer:**
The buffer can grow indefinitely, so the sender never waits.

💡 **Example:**

A printer queue is a bounded buffer—if too many print jobs are queued, new jobs must wait.

Pipes – Simple Data Communication Between Processes

- **Definition:**
A pipe is a unidirectional communication channel that allows data to flow from one process to another.
- **Types:**
 - **Ordinary Pipes:**
Exist only between a parent and child process.
 - **Named Pipes (FIFOs):**
Allow communication between unrelated processes.

💡 Example:

In Linux, the command `ls | grep "file"` creates a pipe, where the output of `ls` is passed as input to `grep`.

Sockets – Synchronization Over a Network

- **Definition:**

A socket is an endpoint for network communication that allows processes on different systems to send and receive messages.

💡 Example:

A web browser and a web server communicate using TCP sockets.

Operating Systems – Lecture 6: Deadlocks

What is a Deadlock?

- **Definition:**

A deadlock occurs when a set of processes become stuck, each waiting for a resource held by another, so that none can proceed.

- **Analogy:**

Imagine two cars on a narrow one-lane bridge coming from opposite directions—neither can move because each waits for the other to back up.

💡 Example:

A system with three processes and three resources where each process holds one resource and waits for the next forms a deadlock.

Four Necessary Conditions for Deadlock

1. **Mutual Exclusion:**

✓ Only one process can use a resource at a time.

2. **Hold and Wait:**

✓ A process holding at least one resource is waiting to acquire additional resources held by others.

3. **No Preemption:**

✓ Resources cannot be forcibly taken from a process; they must be released voluntarily.

4. **Circular Wait:**

✓ A circular chain of processes exists, where each process is waiting for a resource held by the next process in the chain.

💡 Example:

Process P1 holds R1 and waits for R2 (held by P2); P2 holds R2 and waits for R3 (held by P3); P3 holds R3 and waits for R1 (held by P1).

Methods to Handle Deadlocks

1. Deadlock Prevention:

Modify the system to eliminate one or more of the necessary conditions.

- **Mutual Exclusion:**
Where possible, design resources to be shared.
- **Hold and Wait:**
Require processes to request all resources at once.
- **No Preemption:**
Allow resources to be preempted if needed.
- **Circular Wait:**
Impose a strict ordering on resource requests.

Example:

A printer-scanner system might require processes to request the printer before the scanner, preventing circular waiting.

2. Deadlock Avoidance:

Dynamically examine resource-allocation states to ensure that a circular wait can never occur.

- **Banker's Algorithm:**
Each process declares its maximum resource needs; the OS simulates resource allocation and grants requests only if the system remains in a safe state.

Example:

A bank managing loan requests will grant loans only if all customers can eventually be served without a deadlock.

3. Deadlock Recovery:

If a deadlock occurs, the system must detect it and take action to break it.

- **Process Termination:**
Terminate one or more processes to free up resources.
- **Resource Preemption:**
Forcefully take resources away from processes and reassign them.
- **Process Rollback:**
Revert processes to earlier safe states (checkpoints).

Example:

In a database transaction system, processes may rollback to a checkpoint if a deadlock is detected, thereby recovering from the deadlock without losing all progress.

Starvation – The Opposite of Deadlock

- **Definition:**
Starvation occurs when a process waits indefinitely because resources are continuously allocated to higher-priority processes.
- **Solution:**
Aging — Gradually increasing the priority of a waiting process ensures it eventually gets CPU time.

 Example:

A low-priority print job might never print because high-priority jobs keep arriving; aging helps move the low-priority job forward in the queue.

Operating Systems – Lecture 7: Memory Management & Main Memory

Why Memory Management is Important

- **Purpose:**
Memory is a finite resource. The OS must manage it efficiently to ensure smooth process execution and prevent conflicts.
- **Risks without Proper Management:**
Processes may overwrite data or run out of memory if not allocated correctly.

 Example:

A computer with 8GB of RAM running multiple applications (e.g., browser, game, editor, OS services) must allocate memory dynamically without conflicts.

Key Memory Concepts

- **Program Loading:**
The OS loads programs from disk into RAM before execution.
- **Speed Hierarchy:**
 - **CPU Registers:** Fastest storage but very limited.
 - **Main Memory (RAM):** Fast but slower than registers and volatile.
 - **Disk:** Much slower compared to RAM.
- **Cache Memory:**
Acts as an intermediary to reduce the delay between CPU and RAM.
- **Memory Protection:**
Ensures processes do not access unauthorized memory regions.

 Example:

Modern CPUs (e.g., Intel i7) include multiple cache levels (L1, L2, L3) to bridge the speed gap between registers and RAM.

Key Terminology

- **Segment:**
A portion of memory allocated to a process.
- **Physical Address:**
The actual location in RAM.
- **Logical (Virtual) Address:**
The address generated by the CPU, which is mapped to a physical address.
- **Memory Management Unit (MMU):**
Hardware that translates logical addresses into physical addresses.

Example:

A process requesting data from address 0x200A might have it translated by the MMU to physical location 0x3F20A.

Memory Protection & Address Translation

- **Base Register:**
Defines the starting address of a process in memory.
- **Limit Register:**
Specifies the range of memory allocated to a process.
- **Protection:**
The CPU checks every memory access to ensure it stays within the defined base and limit.

Example:

A process with a base address of 1000 and a limit of 4000 can access memory only within that range; any attempt to access outside is blocked by the OS.

Address Binding: When Are Addresses Assigned?

1. **Compile Time:**
 - If the memory location is known beforehand, the compiler generates absolute addresses.
Example: An embedded system with fixed memory allocation.
2. **Load Time:**
 - The OS assigns addresses when loading the program into memory.
Note: Once loaded, the process cannot move.
3. **Execution Time (Dynamic Relocation):**
 - The OS dynamically allocates and moves processes in memory during execution.

💡 Example:

A game running on a PC often requires dynamic relocation because the memory layout changes as new processes start and exit.

Memory Allocation Strategies

- **First-Fit:**
✓ Allocates the first available block that is large enough.
- **Best-Fit:**
✓ Allocates the smallest available block that meets the need.
- **Worst-Fit:**
✓ Allocates the largest available block, leaving the maximum leftover space.

💡 Example:

If there are free blocks of 10MB, 20MB, and 30MB and a process requires 12MB:

- **First-Fit:** May allocate the 20MB block.
 - **Best-Fit:** Chooses the smallest block that fits (likely the 20MB block).
 - **Worst-Fit:** Allocates the 30MB block.
-

Memory Fragmentation: Wasted Space

- **External Fragmentation:**
Occurs when enough total memory is available, but it is not contiguous, preventing allocation.
 - **50% Rule:** For every two allocated blocks, a third block may be lost to fragmentation.
- **Internal Fragmentation:**
Occurs when a process is allocated more memory than it needs, wasting space within that allocation.

💡 Example:

If a process needs 30MB but is allocated 40MB, then 10MB is wasted (internal fragmentation). If 100MB is free but divided into scattered blocks, a 50MB process might not find a contiguous space (external fragmentation).

Swapping – Freeing Memory for Active Processes

- **Concept:**
When RAM is full, the OS moves inactive processes to disk (swapping) and retrieves them when needed.
- **Types:**

- **Static Relocation:**
A swapped-out process returns to the same memory location.
- **Dynamic Relocation:**
A swapped-out process may be loaded into a different location upon return.

 Example:

A paused video editing application is swapped out to free memory and swapped back in when the user resumes work.

Paging – Solving External Fragmentation

- **Definition:**
Paging divides memory into fixed-size blocks (pages) to eliminate external fragmentation.
- **How It Works:**
 - The process is divided into pages.
 - Memory is divided into page frames of equal size.
 - The OS maps pages to available frames, even if non-contiguous.
- **Rule 90/10:**
A process typically uses only 10% of its allocated pages during execution.

 Example:

A 32KB process divided into 4KB pages yields 8 pages, which may be loaded into any available frames in RAM. Paging eliminates external fragmentation but requires a page table to maintain mappings.

Page Tables & Translation Lookaside Buffer (TLB)

- **Page Table:**
A data structure that maps virtual addresses to physical addresses.
- **TLB (Translation Lookaside Buffer):**
A high-speed cache that stores frequently used page table entries to speed up address translation.

 Example:

Without a TLB, each memory access requires multiple lookups; with a TLB, frequently accessed pages are cached, significantly speeding up the process.

Segmentation – An Alternative to Paging

- **Definition:**
Segmentation divides memory into logical segments based on the program's structure.
- **Segments:**
 - **Code Segment:** Contains program instructions.

- **Data Segment:** Stores global variables.
- **Stack Segment:** Manages function calls and local variables.
- **Heap Segment:** Used for dynamically allocated memory.
- **Comparison with Paging:**
 - **Paging:** Uses fixed-size blocks, which can lead to wasted space.
 - **Segmentation:** Uses variable-sized blocks aligned with program structure, reducing internal fragmentation but may require compaction.

💡 Example:

A video editor may store editing functions in the code segment, video clips in the data segment, and temporary effects in the stack segment.

Hybrid Approach: Paging + Segmentation

- **Concept:**
Combining paging and segmentation leverages the benefits of both techniques.
- **Benefits:**
 - The OS allocates only the necessary memory.
 - Processes can use non-contiguous free memory.
 - Two-level mapping allows for better memory utilization.

💡 Example:

A game engine might use segmentation to separate game logic from assets and rendering, while employing paging to efficiently allocate memory without fragmentation.

Operating Systems – Lecture 8: Virtual Memory Management

What is Virtual Memory?

- **Definition:**
Virtual Memory is a technique that separates logical memory from physical memory, giving the illusion of a larger memory space than is physically available.
- **Purpose:**
It allows programs to run even if they do not entirely fit in RAM by swapping data between RAM and disk storage dynamically.

💡 Example:

A computer with 8GB of RAM can run a program that requires 16GB by using virtual memory to swap data between RAM and disk as needed.

Virtual Address Space

- **Concept:**
The virtual address space represents how a process perceives its memory layout, which is different from the physical arrangement in RAM.
- **Implementation Techniques:**
 - **Demand Paging:**
Loads only the required pages of a process into RAM when they are needed.
 - **Demand Segmentation:**
Loads segments of a process on demand rather than fixed-size pages.

💡 Example:

A web browser may load only the visible portions of a webpage initially, fetching additional sections on demand to improve efficiency.

Demand Paging – Efficient Use of RAM

- **Process:**
 - 1. A process requests data from a page not currently in RAM.
 - 2. A page fault occurs, prompting the OS to load the required page from disk into RAM.
 - 3. The process resumes execution after the page is loaded.
- **Advantage:**
Reduces memory consumption by only loading necessary parts of a process.

💡 Example:

Opening a large PDF file may cause the OS to load only the pages you are viewing rather than the entire document.

Page Replacement Strategies – Choosing Which Page to Remove

- **When Required:**
When a page fault occurs and RAM is full, the OS must decide which page to remove to free up space.
- **Common Algorithms:**
 -  **First-In, First-Out (FIFO):**
Removes the oldest page first. Simple but may remove frequently used pages.
 -  **Example:** A browser cache may evict the oldest tabs when memory is full.
 -  **Least Recently Used (LRU):**
Removes the page that has not been accessed for the longest time.
 -  **Example:** A smartphone OS might remove the least-used apps from RAM to free space.
 -  **Least Frequently Used (LFU):**
Removes the page with the fewest accesses.

-  **Example:** A streaming service might cache popular movies while removing rarely watched content.
-  **Random Replacement:**
Removes a random page. Simple but less predictable.
-  **Example:** Used in some real-time systems where precise tracking is not feasible.

Importance of Virtual Memory

- **Enables Large Applications:**
Programs can run on systems with less physical RAM than they require.
- **Improves Multitasking:**
Allows efficient management of memory allocation among multiple processes.
- **Prevents Fragmentation:**
Uses paging or segmentation techniques to avoid memory fragmentation issues.

Real-World Use Case:

Gaming consoles and cloud servers often use virtual memory to dynamically allocate resources based on demand.

Operating Systems – Lecture 9: Storage System Structure

How Does a Disk Work?

- **Components:**
 - **Platters:** Disks consist of multiple platters with read/write heads.
 - **Tracks & Sectors:**
 - ✓ Data is stored on circular tracks divided into sectors.
 - ✓ A group of tracks across platters forms a cylinder.
- **Key Factors Affecting Disk Access Time:**
 - **Seek Time:**
The time for the disk head to move to the correct track.
 - **Rotational Latency:**
The time for the correct sector to rotate under the read/write head.
 - **Data Transfer Time:**
The time taken to transfer data to/from the disk.

Example:

When you open a file, the OS locates its storage location, moves the disk head to the correct track, waits for the sector to rotate into place, and then reads the data into RAM.

Disk Scheduling Algorithms

- **Purpose:**
To optimize the order in which disk I/O requests are serviced, minimizing seek time and improving performance.
- **Common Algorithms:**
 -  **First-Come, First-Served (FCFS):**
Processes requests in the order they arrive.
 - **Advantage:** Fair.
 - **Disadvantage:** May result in long seek times.
 -  **Example:**
Requests arriving in the order: 98, 183, 37, 122, 14, 124, 65, 67 can lead to high head movement.
 -  **Shortest Seek Time First (SSTF):**
Selects the request closest to the current head position.
 - **Advantage:** Reduces seek time.
 - **Disadvantage:** May cause starvation for far-off requests.
 -  **Example:**
With the head at 50 and requests at 10, 20, 60, 80, SSTF might choose 60 first, then 80, followed by 20, and 10.
 -  **SCAN (Elevator Algorithm):**
The disk head moves in one direction, servicing requests along the way, then reverses direction.
 - **Advantage:** Prevents starvation by servicing requests in order.
 -  **Example:**
With requests at 10, 50, 180, 190 and the head starting at 100, the head moves, for example, towards 50 then 10, then reverses to handle 180 and 190.
 -  **C-SCAN (Circular SCAN):**
The disk head moves in one direction, and upon reaching the end, it quickly returns to the beginning to service requests again.
 - **Advantage:** Provides more uniform wait times.
 -  **Example:**
The head moves from 100 to 200, resets to 0, and continues scanning, ensuring fair access to all requests.

Storage Optimization Techniques

- **Interleaving (Non-Sequential Allocation):**
 - ◆ Blocks are stored non-sequentially and spaced apart to match the disk's access speeds, preventing bottlenecks when reading large files.
- **Read Ahead (Prefetching):**
 - ◆ The disk controller anticipates upcoming block requests and loads them into cache before they are requested.

💡 **Example:**

A video player may preload the next few seconds of a video to prevent buffering interruptions.

Error Detection & Correction in Storage

- **Techniques:**
 - **Parity Bit:**
 - ◆ Adds an extra bit to detect errors; effective for detecting single-bit errors but cannot correct them.
 - **Cyclic Redundancy Check (CRC):**
 - ◆ Uses polynomial-based hash functions to detect data corruption with higher accuracy.
 - **Error Correction Code (ECC):**
 - ◆ Detects and corrects small (soft) errors automatically; hard errors (physical damage) can usually only be detected.

💡 **Example:**

ECC memory in servers automatically corrects small errors to prevent system crashes.

Storage Formatting & File System Management

- **Low-Level Formatting:**
 - ✓ Divides the disk into sectors, preparing it for data storage.
- **Root Partition:**
 - ✓ Contains the operating system and essential files.
- **Swap Space:**
 - ✓ A designated disk area used for paging and virtual memory operations.

💡 **Example:**

When RAM is full, inactive data is moved to swap space to free up memory for active processes.

Network Storage & Redundancy

- **Network-Attached Storage (NAS):**
 - ◆ Allows multiple devices to access shared storage over a network using protocols like NFS and CIFS.

- **Redundant Array of Independent Disks (RAID):**
 - ◆ Combines multiple disks to improve performance and fault tolerance.
 - **RAID 0:**
Striping for speed, no redundancy.
 - **RAID 1:**
Mirroring for data protection.
 - **RAID 5:**
Striping with parity for fault tolerance.
- **Snapshot:**
 - ◆ Captures the state of a file system at a specific point, used for backups and quick recovery.

💡 Example:

Before a major update, a snapshot of the OS can be taken so that the system can roll back if something fails.

Operating Systems – Lecture 10: File System Interface & Implementation

User Data Access Requirements

- **Hardware Provides:**
 - **Durability:**
Data remains stored even after power loss.
 - **Speed:**
Fast read/write operations.
- **The OS Ensures:**
 - **Storage Capacity:**
Efficient allocation of disk space.
 - **Access Control & Security:**
Restricting unauthorized access.
 - **Ease of Use:**
Providing a structured interface for file operations.

💡 Example:

A database system requires fast, reliable, and secure access, while a log file system needs continuous writing capability.

What is a File?

- **Definition:**
A file is a logical unit of storage on a disk containing data or programs.

- **File Attributes:**

- **Name:** Unique identifier.
- **Type:** E.g., text, binary, executable.
- **Location:** Disk address where the file is stored.
- **Size:** File length in bytes.
- **Permissions:** Who can read, write, or execute.
- **Timestamps:** Creation, modification, and last accessed times.

💡 **Example:**

A Word document has attributes like name (report.docx), type (text), size (2MB), and last modified time.

Common File Operations (System Calls)

- **Data Operations:**

- **create():**
Create a new file.
- **open():**
Open an existing file.
- **read():**
Read data from a file.
- **write():**
Write data to a file.
- **delete():**
Remove a file.
- **close():**
Close an opened file.
- **seek():**
Move the file pointer to a specific location.

- **Metadata Operations:**

- Retrieve and update file attributes (size, timestamps, permissions).

💡 **Example:**

Media players use seek() to jump to a specific part of a video, and the OS maintains metadata for security purposes.

Open File Tables – Managing Open Files

- **Global Open File Table:**

- ✓ Shared by all processes.
- ✓ Stores attributes of all opened files and tracks disk locations.
- ✓ Contains file pointers that indicate the current read/write positions.
- **Per-Process Open File Table:**
 - ✓ Each process has its own table with entries pointing to the global open file table.
 - ✓ Tracks process-specific file access mode (read, write, execute).

💡 Example:

When two programs open the same file, the global table maintains consistency, while each process has its own file pointer.

File Access Methods

- **Sequential Access:**
 - ◆ Data is processed in order (byte by byte).
 - ◆ Ideal for text files, logs, and media streaming.

💡 Example:

Reading a book from the first page to the last.

- **Direct (Random) Access:**
 - ◆ Data can be accessed at any specific location using an offset.
 - ◆ Suitable for databases and indexed files.

💡 Example:

Jumping directly to a specific scene in a movie file.

Directory Structures

- **Single-Level Directory:**
 - ✓ All files are stored in a single directory.
 - ✓ Simple but can lack organization.

💡 Example:

Early MS-DOS systems used a flat file system.

- **Two-Level Directory:**
 - ✓ Each user has their own directory to prevent filename conflicts.

💡 Example:

Windows User Profiles (e.g., C:\Users\Alice\Documents vs. C:\Users\Bob\Documents).

- **Multi-Level Directory (Hierarchical):**

- ✓ Uses a tree-like structure with subdirectories for better organization.

💡 **Example:**

UNIX/Linux file system structure (e.g., /home/user/Documents/notes.txt).

- **Links:**

- **Hard Link:**

Points directly to the file's inode, acting as a second name.

- **Soft Link (Symbolic Link):**

Points to a file name and can exist across directories.

💡 **Example:**

Hard links are often used for backups, while soft links serve as shortcuts.

Disk Performance & Efficiency Metrics

- **Disk Overhead:**

- ✓ The processing time the CPU spends handling a disk request.

- **Disk Latency (Access Time):**

- **Seek Time:**

Time for the disk head to move to the correct track.

- **Rotation Time:**

Time for the sector to rotate under the disk head.

- **Transfer Rate:**

- ✓ The speed at which data is transferred from disk to memory.

💡 **Example:**

SSDs have lower seek times and rotational delays compared to HDDs because they have no moving parts.

File System Implementation & Memory Similarities

- **Continuous Allocation:**

- ✓ Allocates contiguous blocks of disk space.

- **Drawback:** Can lead to external fragmentation, requiring defragmentation.

- **Indexed Allocation:**

- ✓ Uses an index table (similar to paging) to map file blocks to disk locations.

- **Free Space Management:**

- **Bitmap:**

Uses a bit array to track free and occupied blocks.

- **Linked List:**
Maintains a list of free blocks.

💡 **Example:**

Windows NTFS may use bitmaps, while Linux ext4 uses a combination of bitmaps and linked lists.

Operating Systems – Lecture 11: Virtualization & Cloud Computing

What is Virtualization?

- **Definition:**
Virtualization is the process of creating a virtual (software-based) version of a physical resource—such as an operating system, storage, or network—allowing multiple independent environments to run on a single physical system.
- **Types of Virtualization:**
 - **Type 1 / Bare Metal Hypervisor:**
 - ✓ Runs directly on the hardware and manages virtual machines without an underlying OS.
 - 💡 **Example:** Cloud providers like AWS and Google Cloud use Type 1 hypervisors (e.g., VMware ESXi, Xen) to host thousands of VMs.
 - **Type 2 / Hosted Hypervisor:**
 - ✓ Runs on top of an existing OS (e.g., VMware Workstation, VirtualBox).
 - 💡 **Example:** Developers often use Type 2 hypervisors on their laptops for testing environments.

What is a Hypervisor?

- **Definition:**
A Hypervisor (Virtual Machine Manager - VMM) is a software layer that enables multiple operating systems (OS) to share the same hardware efficiently.
- **Responsibilities:**
 - Allocates CPU, memory, and storage to virtual machines.
 - Ensures isolation between VMs to prevent conflicts.
 - Supports live migration (moving a VM between physical machines).

💡 **Example:**

KVM, VMware ESXi, Microsoft Hyper-V, and Xen are common hypervisors used in enterprise cloud computing.

What is Cloud Computing?

- **Definition:**
Cloud Computing is an on-demand service model that provides scalable computing resources (servers, storage, and applications) over the internet.
- **Key Attributes:**
 - **Scalability:**
Dynamically increases or decreases resources based on demand.
 - **Low Latency:**
Provides fast response times for user requests.
 - **Guaranteed Performance:**
Ensures consistent resource availability.
 - **Extensible Management:**
Can be monitored and controlled remotely.
 - **Self-Healing Resilience:**
Automatically detects and recovers from failures.

 Example:

Netflix uses cloud services to scale video streaming globally while ensuring smooth performance.

Essential Characteristics of Cloud Computing

- **On-Demand Self-Service:**
✓ Users can automatically provision computing resources without human intervention.
 **Example:** AWS EC2 instances can be launched and terminated at any time.
 - **Broad Network Access:**
✓ Services are accessible from any device over the internet.
 **Example:** Google Drive allows file access from desktops, tablets, and smartphones.
 - **Resource Pooling:**
✓ Cloud providers dynamically allocate resources to multiple customers.  **Example:** AWS dynamically assigns server resources based on customer workloads.
 - **Rapid Elasticity:**
✓ Resources can be scaled up or down automatically.  **Example:** A retail website increases cloud server capacity during Black Friday sales.
 - **Measured Service:**
✓ Customers pay based on actual resource usage.  **Example:** AWS charges for storage (S3) and computing power (EC2) based on usage time.
-

Cloud Service Models

- **Infrastructure as a Service (IaaS):**
 - ◆ Provides virtualized computing resources over the internet.
 - 💡 **Example:** AWS EC2, Google Compute Engine, Microsoft Azure VMs.
- **Platform as a Service (PaaS):**
 - ◆ Offers a ready-to-use development environment (databases, application servers, middleware) without managing the underlying infrastructure.
 - 💡 **Example:** Google App Engine, AWS Lambda, Microsoft Azure App Services.
- **Software as a Service (SaaS):**
 - ◆ Delivers fully functional applications over the internet, eliminating the need for users to install or maintain software.
 - 💡 **Example:** Google Workspace (Docs, Gmail), Microsoft 365, Dropbox.